In-Core Zicsr Interrupt Subsystem with Round-Robin Anti-Starvation for a 5-Stage RV32IM Core

Paper-style proposal · figures and tables are monochrome · values marked "est." or "target" are not measured results.

Contents. Table 1 literature comparison · Fig 1 SoC · Fig 2 current fixed-priority IRQ path · Fig 3 proposed round-robin path · Table 2 flop budget · Table 3 configurability.

☐ existing block ■ new block ▨ problem area ⬚ optional / other milestone

1 · Introduction and motivation

The RISCFORGE core takes interrupts straight from peripheral wires into `mip` and picks a winner with a fixed-priority chain. That is fast but has four weaknesses (Fig. 2): transient pulses can be lost, a busy high-ranked source can starve lower ones and the system tick, the threshold compares cause numbers instead of priorities, and a divide in EX can delay acceptance. We propose a small core-local front end that fixes these in standard CSR space, with no bus traffic and no compiler changes.

2 · Literature review

Design	Interface	Priority	Levels / nesting	Fairness	Dispatch	Cost note
PicoRV32 IRQ [3]	Custom instructions (`getq`, `setq`, `retirq`); 4 extra q-registers by default	Software, from a bitmask of active IRQs	None	Software	Single entry, software decode	Size-optimised core, CPI 4–5, so cycles are not comparable
Ibex [2]	Standard privileged 1.11 CSRs	Fixed: fast lines first, lowest ID first	None	None	Vectored, base + 4×ID	15 fast lines + 3 standard + NMI
PLIC [4]	Memory-mapped claim / complete	Per-source priority + threshold	Threshold only	None	Software via claim	Bus round trip per claim and complete
PULP CLIC [1, 5]	CSRs + memory-mapped config	Level, then priority, then ID	Yes	None	Selective HW vectoring, `xnxti`	About 176 GE per source in config registers [1]
CV32RT fastirq [1]	CLIC + banked registers	As CLIC	Yes	None	HW vectoring + background save	+36% register file, +21% ID stage [1]
This work	**Standard vectored `mtvec` + custom CSRs 0x7C0–0x7C2**	**4 classes in hardware**	**Yes (level stack)**	**Round-robin in class, formally bounded**	**HW vectoring**	**About 110 flops (est., target)**

Table 1 — Interrupt front ends compared. PicoRV32 wins on raw area but uses a non-standard interface with software priority, so the claim here is capability per gate, not smallest area. Ibex confirms fixed priority is the norm in small open

cores.

3 · System context

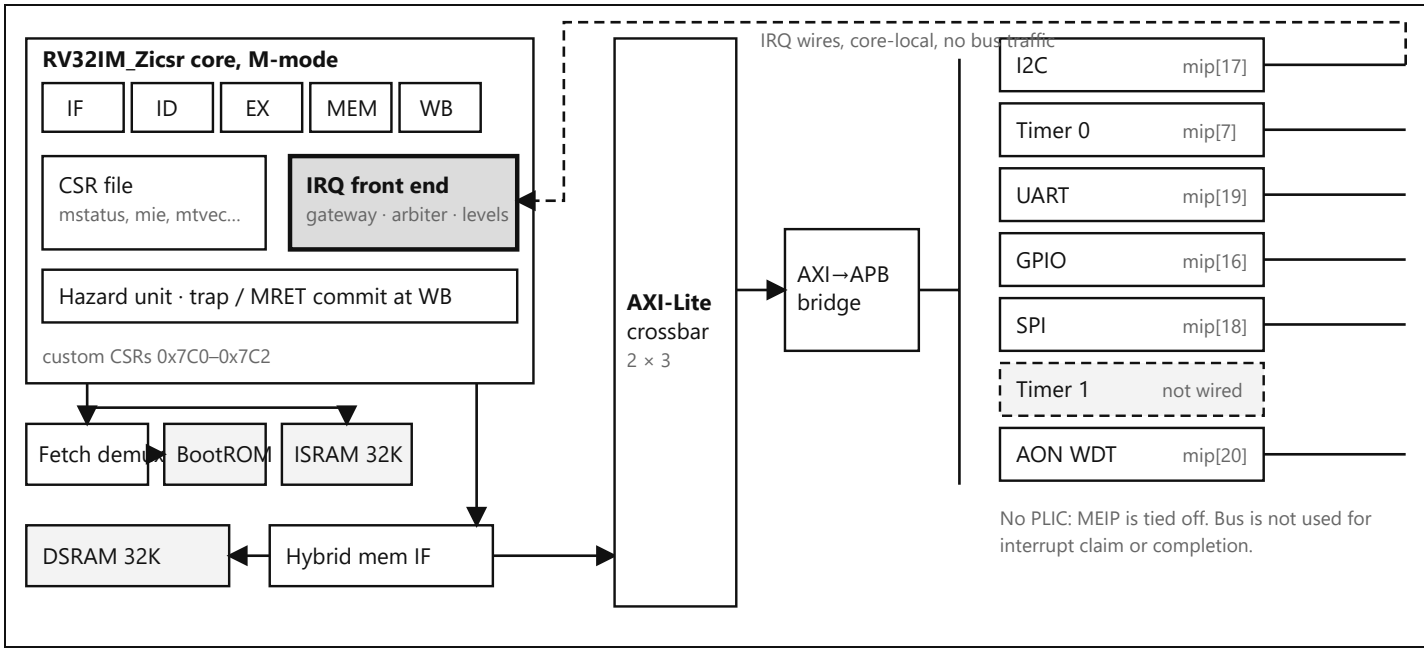


Fig. 1 — RISCFORGE SoC. Instruction fetch and DSRAM data have direct paths; only peripheral registers cross the crossbar and bridge. Interrupt wires (dashed) go from peripherals to the core's IRQ front end. Per the SoC document; confirm the WDT ordering against the RTL.

4 · Current implementation: fixed priority

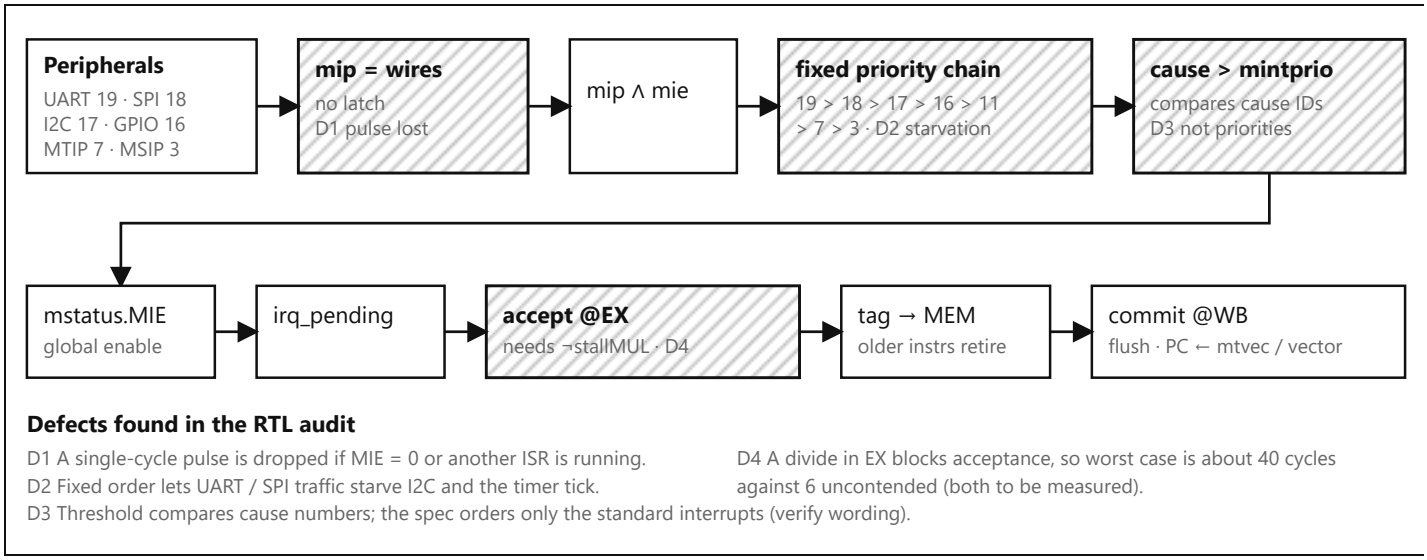


Fig. 2 — Current path: wires straight into `mip`, one fixed chain, then EX accept and WB commit. Hatched blocks carry a defect.

5 · Proposed implementation: round-robin with classes

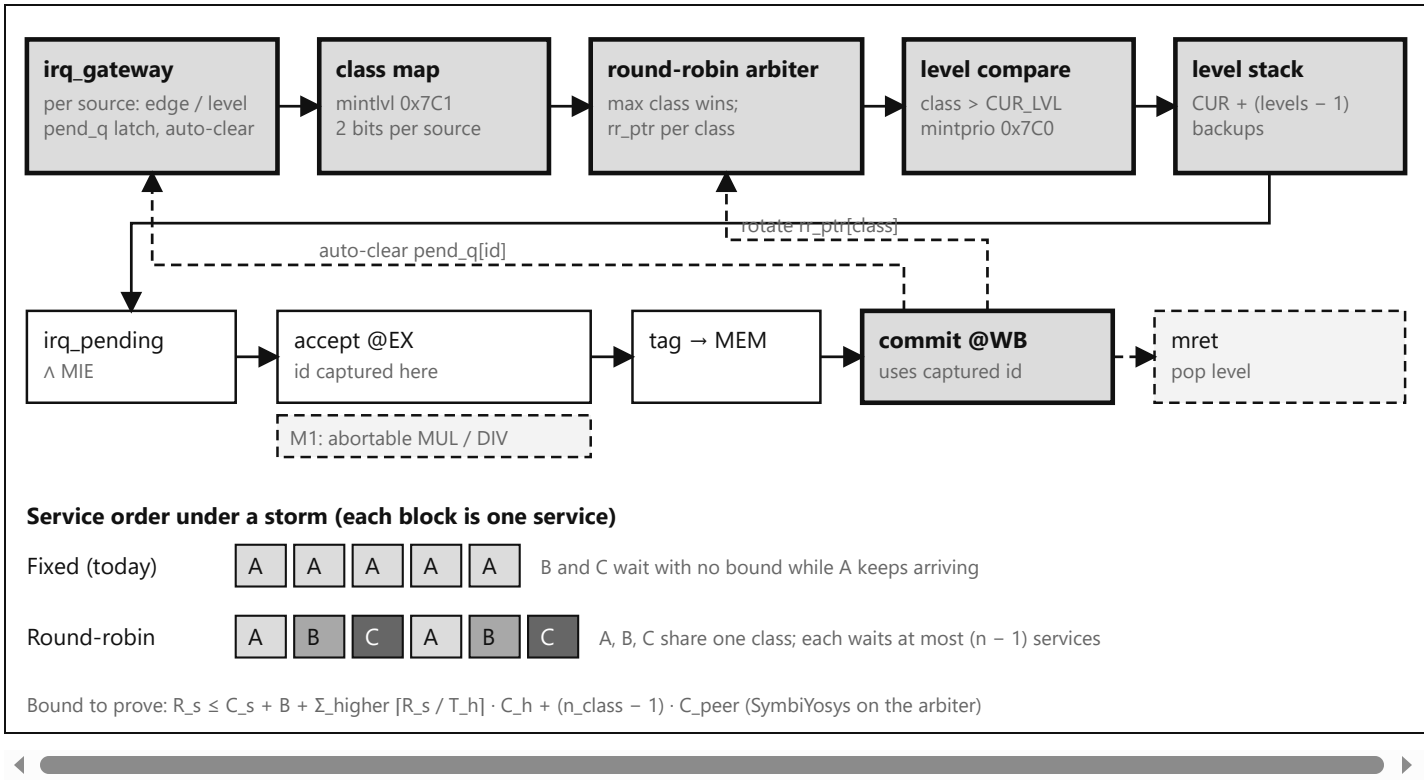


Fig. 3 — Proposed path. Solid blocks in the top row are new; dashed arrows are commit-time feedback (auto-clear and pointer rotation). The pointer moves at commit so a cancelled accept never disturbs fairness.

How the solution works

- **Gateway:** a rising edge sets `pend_q`, so a pulse survives while MIE is clear or another ISR runs. Commit clears the bit for the captured id, and a new pulse in the same cycle wins over the clear.
- **Classes and round-robin:** urgent sources sit in a higher class and still win. Inside a class the pointer moves past the last served source, so no peer is starved.
- **Levels:** an interrupt preempts only if its class is above the current level; hardware pushes on entry and pops on `mret`. Depth is levels – 1 backups, so 4 classes need 3.
- **Toolchain:** the standard vectored `mtvec` and `mret` are unchanged. Handlers still save `mepc` before re-enabling MIE for nesting.

6 · Area and configurability

Feature added (cumulative)	State	Flops (est.)
P0: mask + pending only (PicoRV32-style baseline)	existing `mie`, wires	0 new
+ Edge / level gateway	16 pend_q + 16 mode bits	32
+ Priority classes	2 bits × 16 sources	32
+ Round-robin pointers	4 classes × 4 bits	16
+ Level stack	CUR 2 bits + 3 backups × 2 bits	8
Total state (plus about 20 control flops)		**88 → about 110**

Table 2 — Flop budget per feature, built up from a PicoRV32-like minimum. Gate equivalents come from synthesis (Yosys on an open PDK) and will be reported per feature. The CLIC comparison is a 16-source PULP CLIC synthesized in the same flow.

Parameter	Range	Set at	Effect
Number of sources N	8, 16, 32	Elaboration	About 4 flops per source plus arbiter logic
Number of classes L	2–8	Elaboration	Stack depth L − 1; pointer count L
Class of each source	0–(L − 1)	Runtime CSR 0x7C1	Firmware sets urgency
Edge or level per source	1 bit	Runtime CSR 0x7C2	Level sources still need handler clear
Round-robin on or off	1 bit	Elaboration	Off gives fixed priority inside a class

Table 3 — Configuration. Unlike a CLIC, there is no per-source memory-mapped register, so area grows by a few flops per source instead of a 32-bit register (about 176 GE per source in [1]).

7 · Evaluation and validation

- **Formal (SymbiYosys):** pulse never lost, no lower class granted before a waiting higher class, same-class wait at most n − 1 services, auto-clear only for the taken id. Baseline gives a counterexample; the new design proves it.
- **Metrics:** flops and GE per feature, worst-case wait per source, acceptance latency (with the abortable divider: about 40 → 6, to confirm), RISCOF still passing.
- **Venues:** 4–5 page letter or conference first; TCAS-II only with the divider bound.

References

1. R. Balas, A. Ottaviano, L. Benini, "CV32RT: Enabling Fast Interrupt and Context Switching for RISC-V Microcontrollers," IEEE TVLSI, 2024.
2. lowRISC, Ibex documentation, "Exceptions and Interrupts," ibex-core.readthedocs.io.
3. YosysHQ, PicoRV32 README, "Custom Instructions for IRQ Handling," github.com/YosysHQ/picorv32.
4. RISC-V International, Privileged Architecture and PLIC specifications.
5. RISC-V, Smclic CLIC draft, github.com/riscv/riscv-fast-interrupt; PULP CLIC, github.com/pulp-platform/clic.